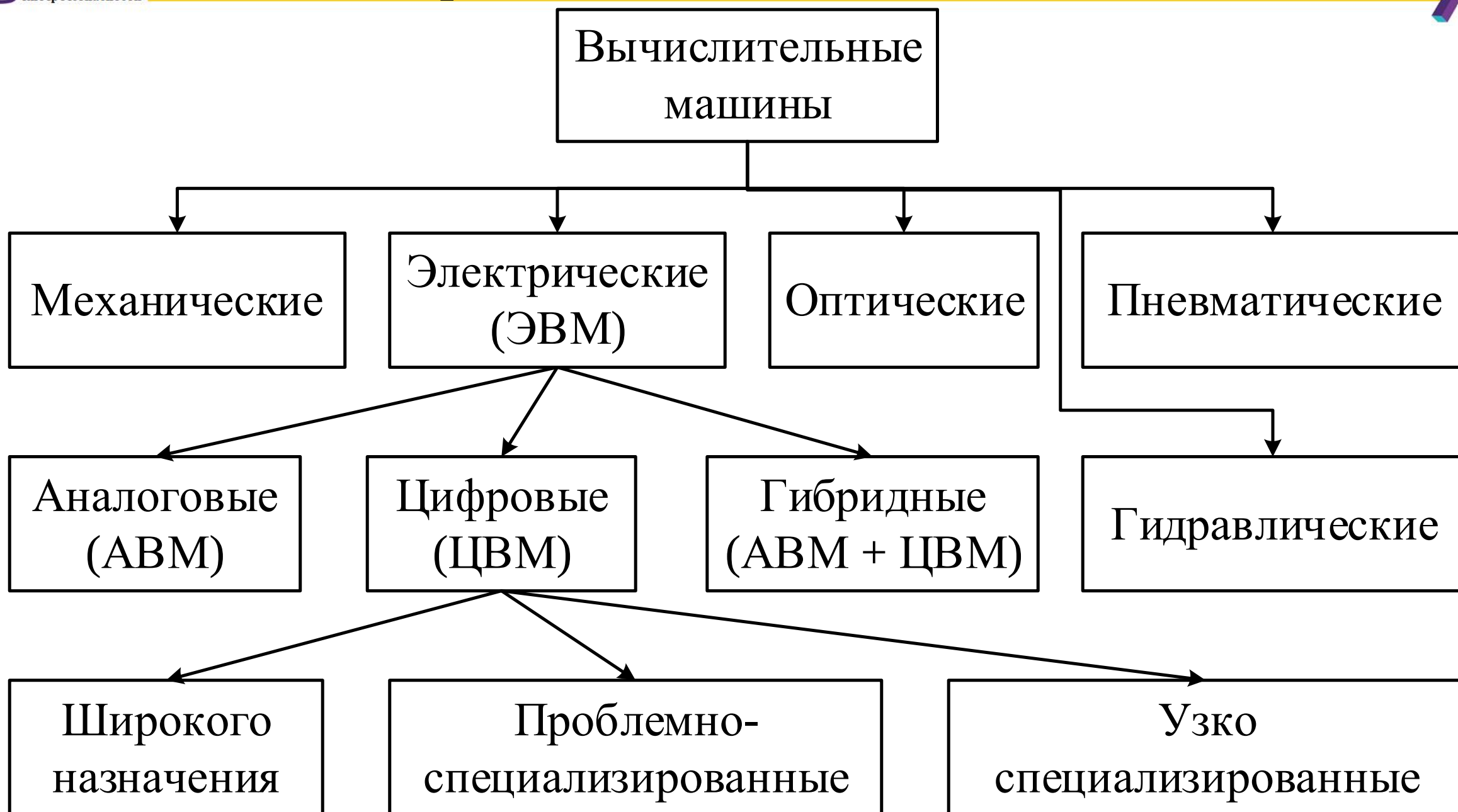


ЛЕКЦИЯ 4



ОСНОВЫ ПОСТРОЕНИЯ ЭЛЕКТРОННЫХ ВЫЧИСЛИТЕЛЬНЫХ МАШИН

- Классификация ВМ
- Типовые архитектуры ЭВМ
- Принципы фон Неймана, абстрактное центральное устройство ЭВМ
- Основы построения центрального процессора
- Функциональные блоки центрального процессора
- Командный цикл процессора





Вычислительная система (ВС) – это совокупность взаимосвязанных и взаимодействующих процессоров или вычислительных машин, периферийного оборудования и программного обеспечения.

Электронные вычислительные машины и системы:

- Обычные вычислительные машины (ЭВМ)
- Суперкомпьютеры (многопроцессорные ЭВМ)
- Вычислительные системы

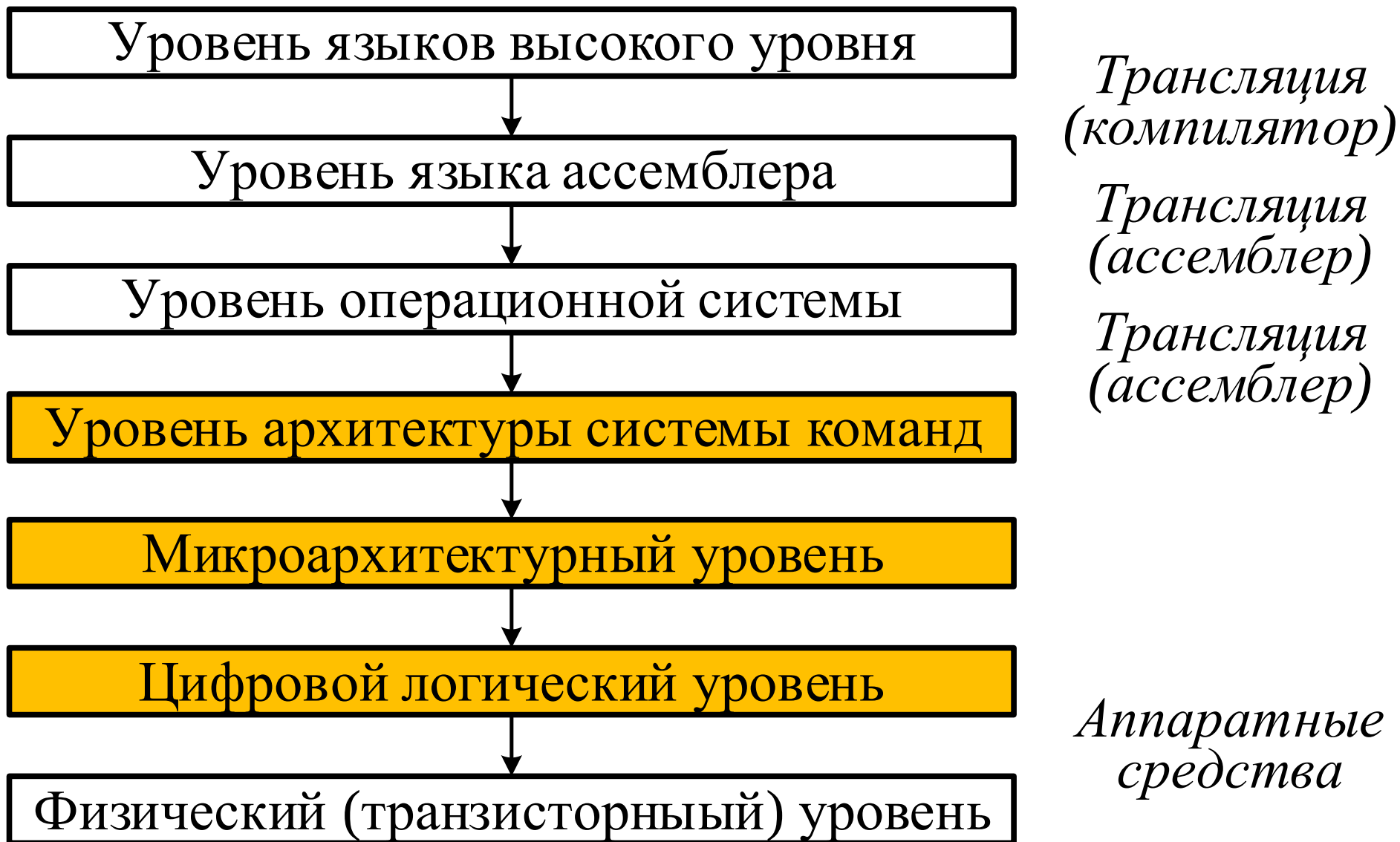
Отличие ВС от ЭВМ состоит в количестве вычислителей.



- **Функциональная организация ВМ** – абстрактная модель совокупности функциональных возможностей и услуг, призванных удовлетворить потребности пользователей.
- Согласно ГОСТ 15971-90: **архитектура ВМ** – это концептуальная структура ВМ, определяющая проведение обработки информации и включающая методы преобразования информации в данные и принципы взаимодействия технических средств и программного обеспечения.
- Согласно ISO/IEC 2382:2015: **архитектура ВМ** – это логическая структура и функциональные характеристики ВМ, включая взаимосвязи между её аппаратными и программными компонентами.



- **Структурная организация ВМ** – это физическая модель, которая устанавливает состав, порядок и принципы взаимодействия основных функциональных частей машины.
- **Структура ВМ** – это совокупность его функциональных элементов и связей между ними. Элементами могут быть различные устройства – от элементарных логических узлов до простейших схем. Структура описывает технические, программные и программно-аппаратные средства.
- **Структурная схема** – это графическое отображение структурной организации на некотором уровне детализации.



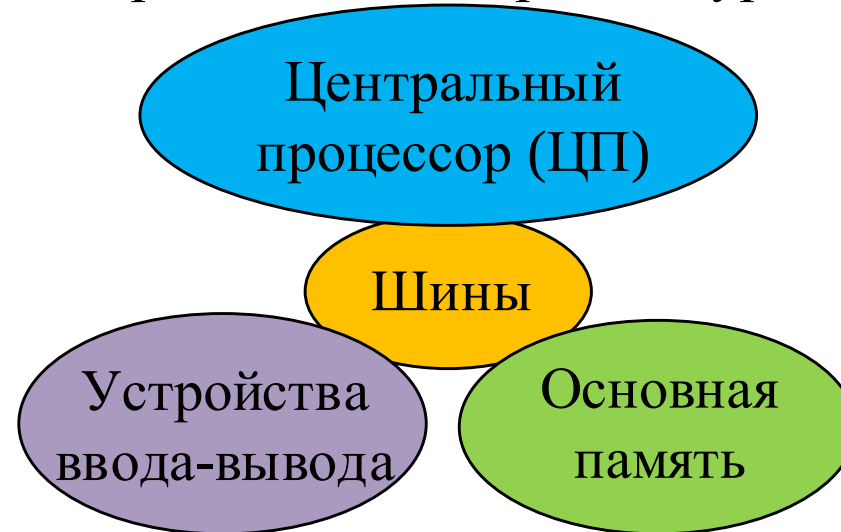


Специалист	Круг вопросов
Производитель полупроводниковых материалов	Материал для интегральных микросхем (легированный кремний, диоксид кремния...)
Разработчик электронных схем	Электронные схемы узлов ВМ (разработка и анализ)
Разработчик интегральных микросхем	Сверхбольшие интегральные микросхемы (схемы электронных элементов, их размещение на кристалле)
Системный архитектор	Архитектура и организация вычислительной машины (устройства и узлы, система команд,...)
Системный программист	Операционная система, компиляторы
Теоретик	Алгоритмы, абстрактные структуры данных

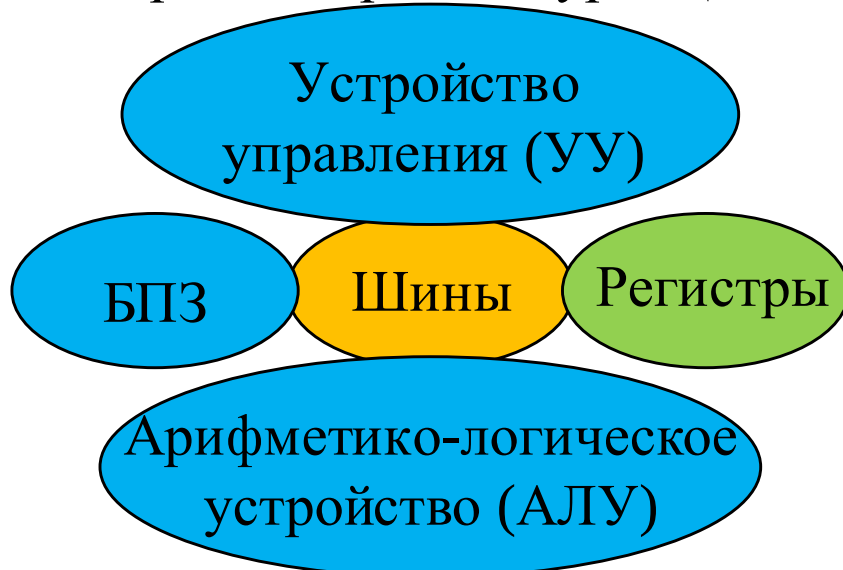
Уровень «чёрного ящика»
Входы Выходы



Уровень общей архитектуры



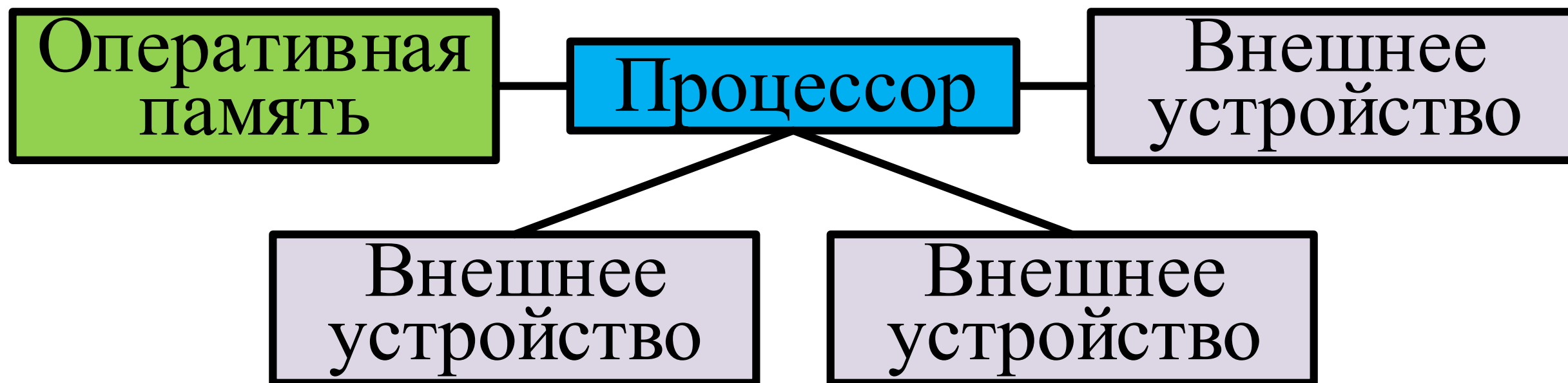
Уровень архитектуры ЦП

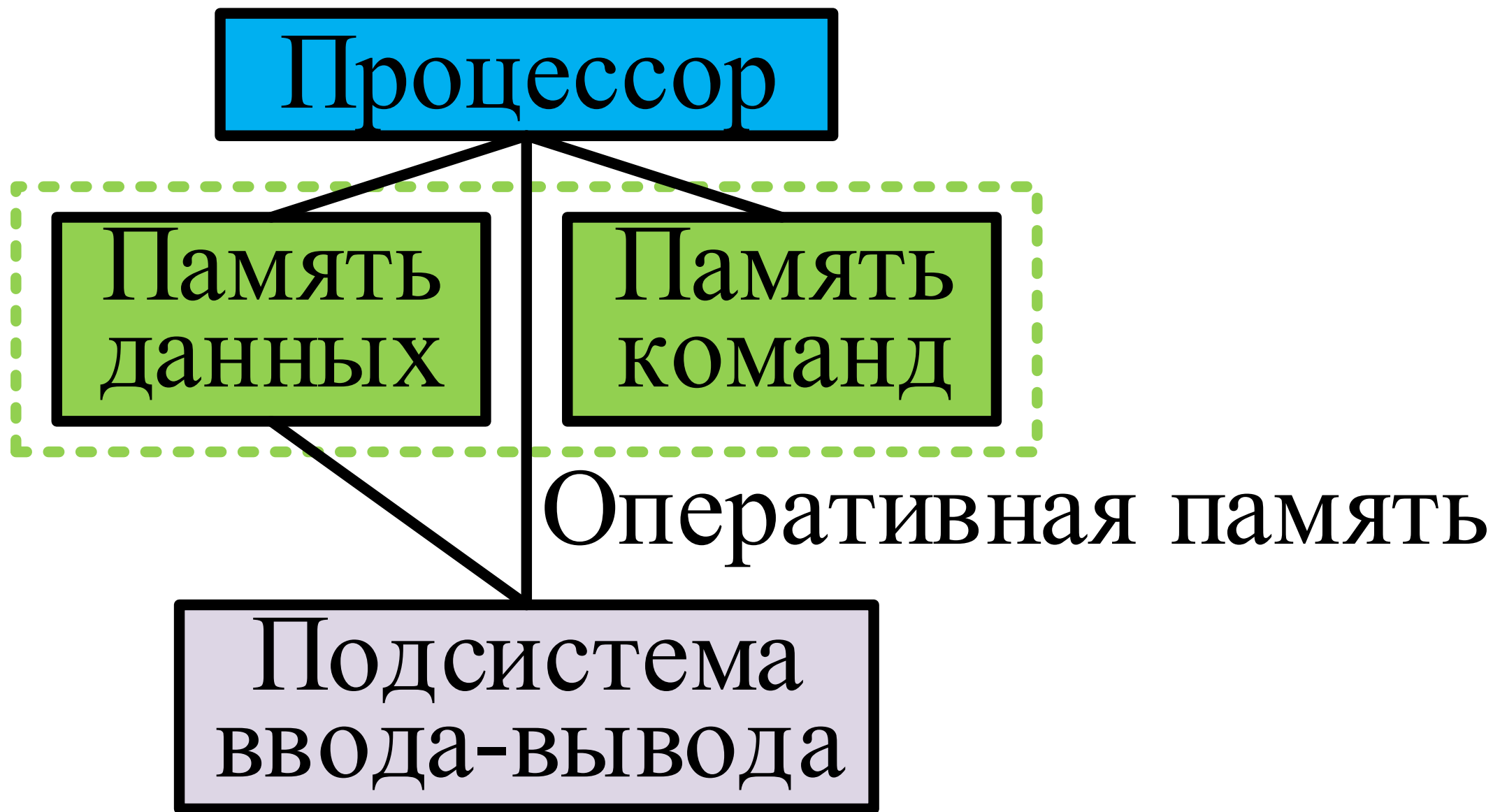


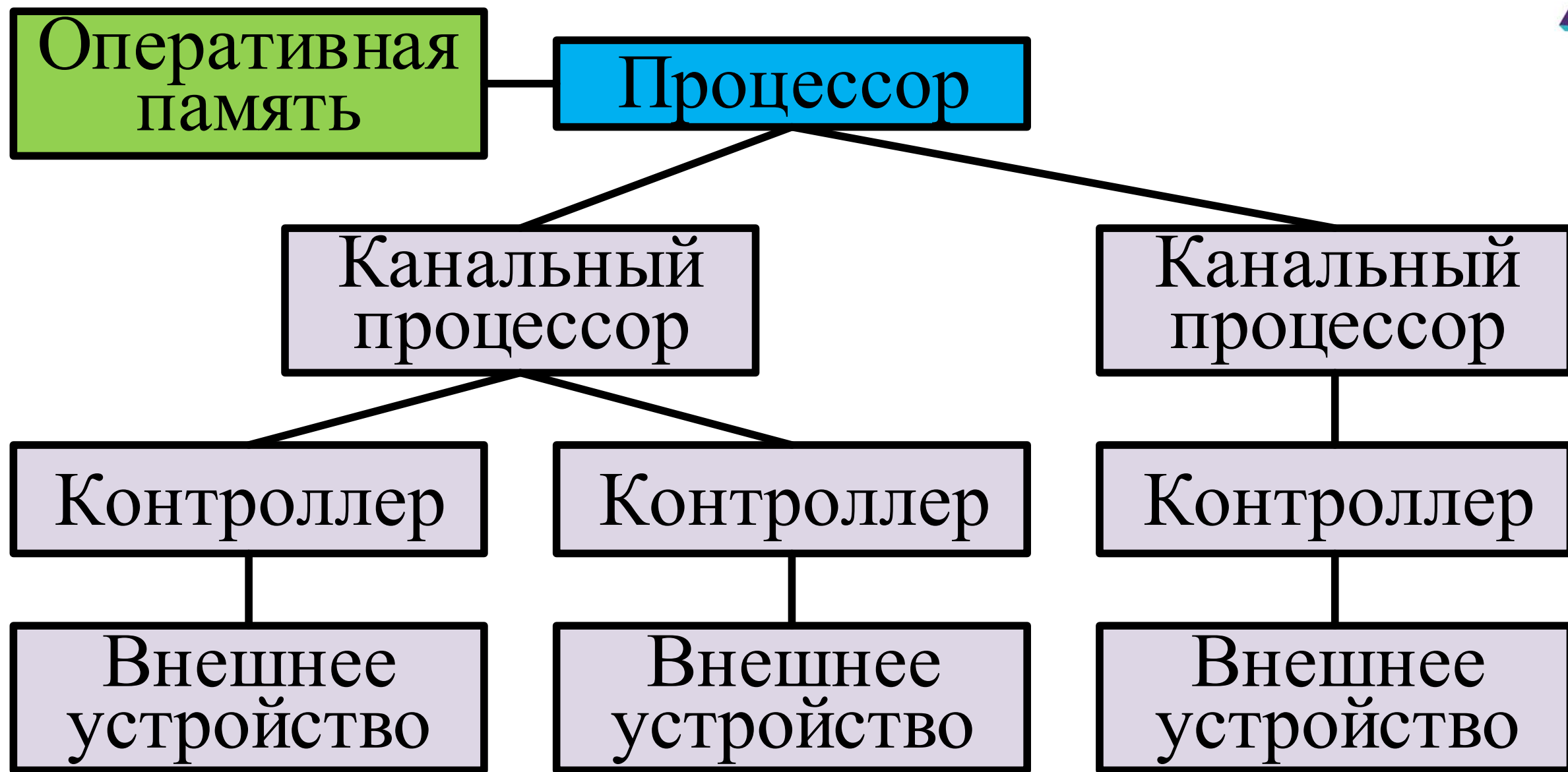
Уровень архитектуры УУ

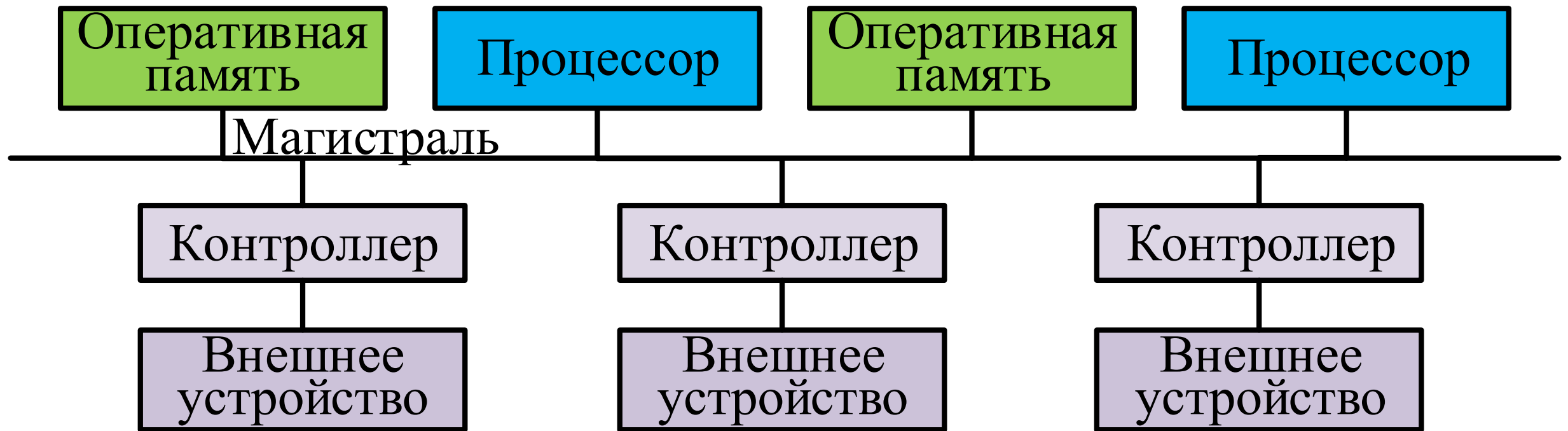


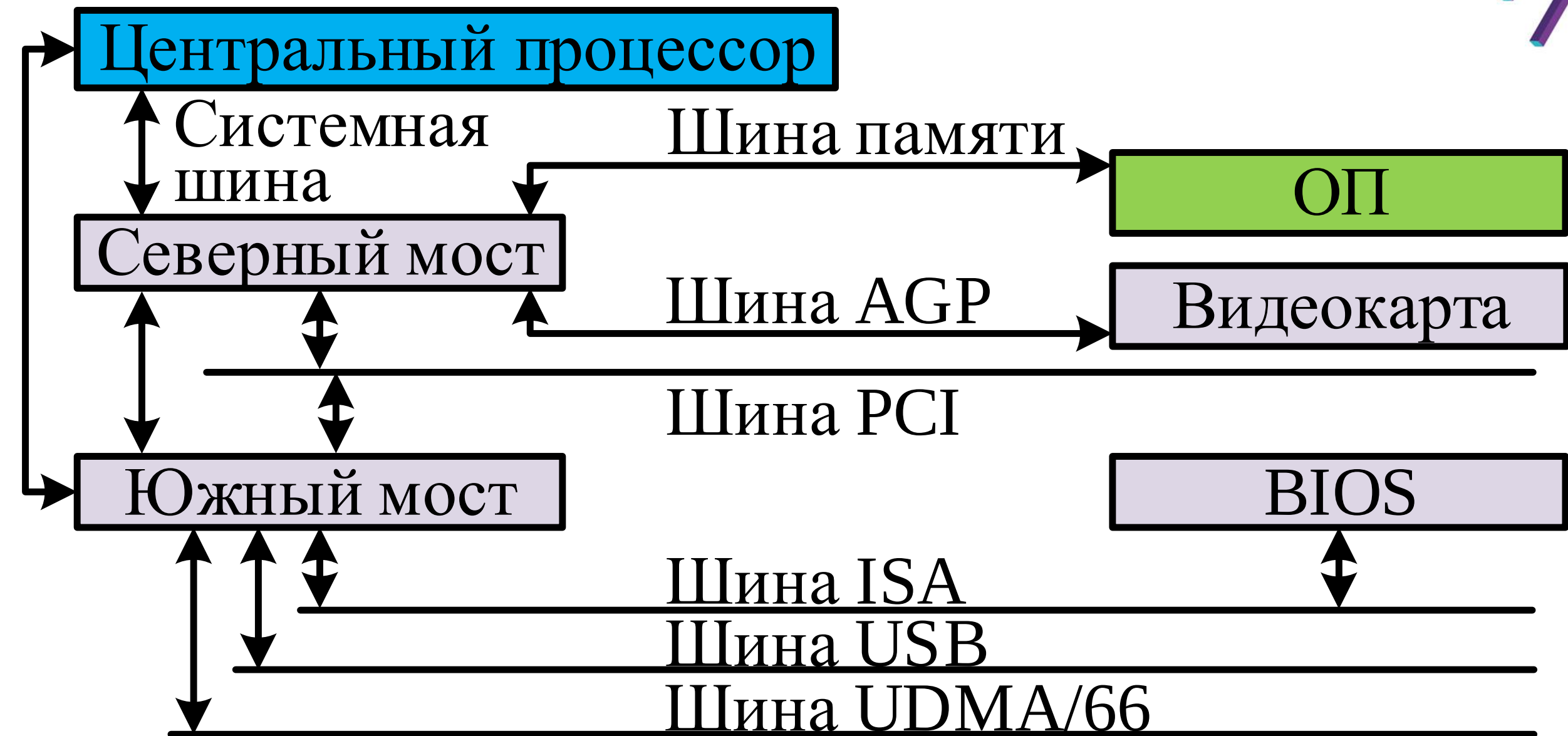
Тип архитектуры	Характеристики	Примеры архитектур
ЭВМ в целом	Взаимосвязь компонентов	Иерархическая, магистральная
Центральное устройство	Использование оперативной памяти	Принстонская, гарвардская
Вычислительная система	Взаимосвязь компонентов	Кластерная, массивно-параллельная
Микроархитектура процессора	Обработка микроопераций, кэш-память, конвейеры	AMD K7-K8, Intel P5-P6
Система команд	Команды процессора	IA-32, IA-64, AMD64
Микроархитектура чипсета	Взаимосвязь компонентов системной платы ПК	«Северный мост – Южный мост»



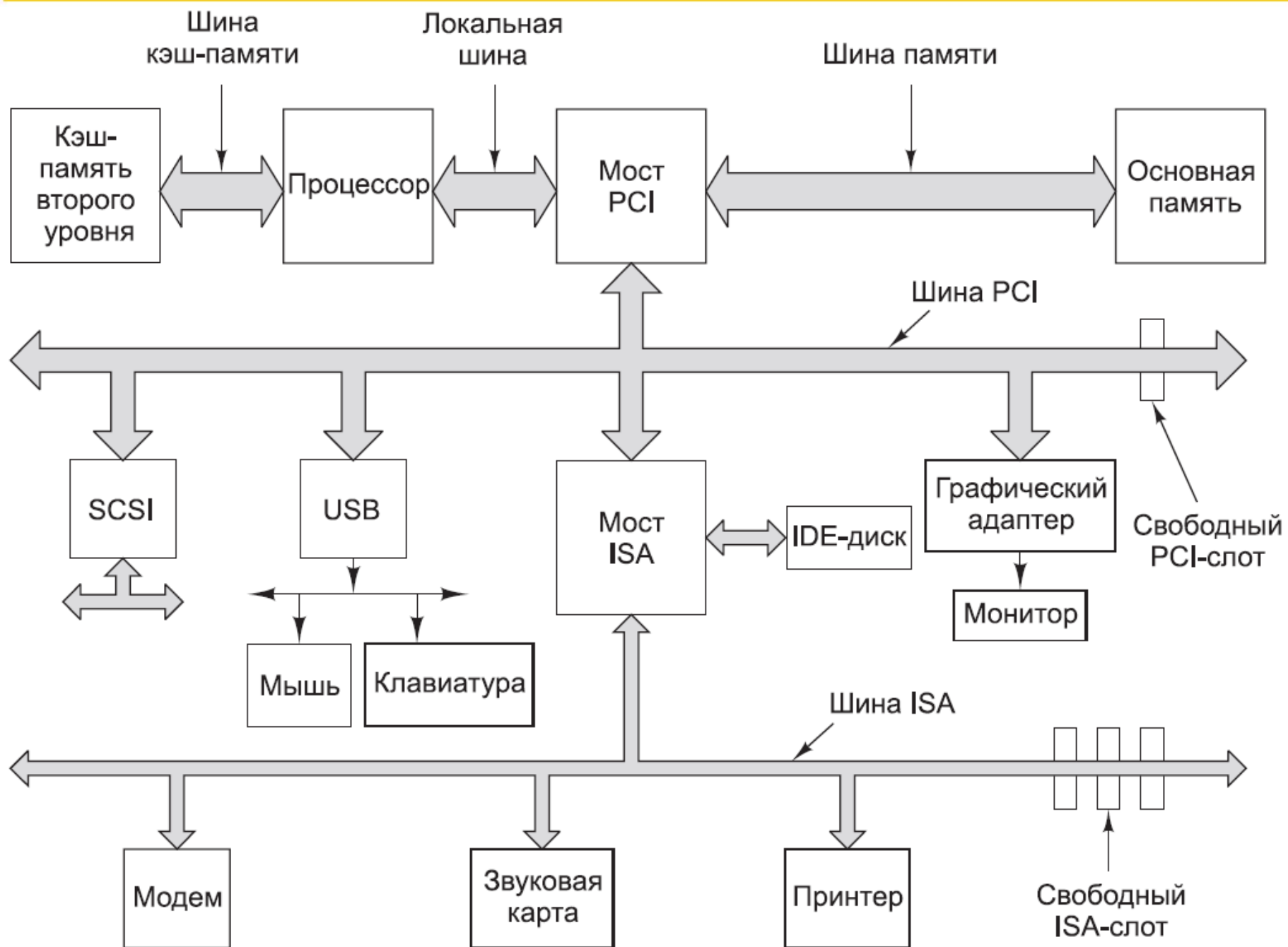




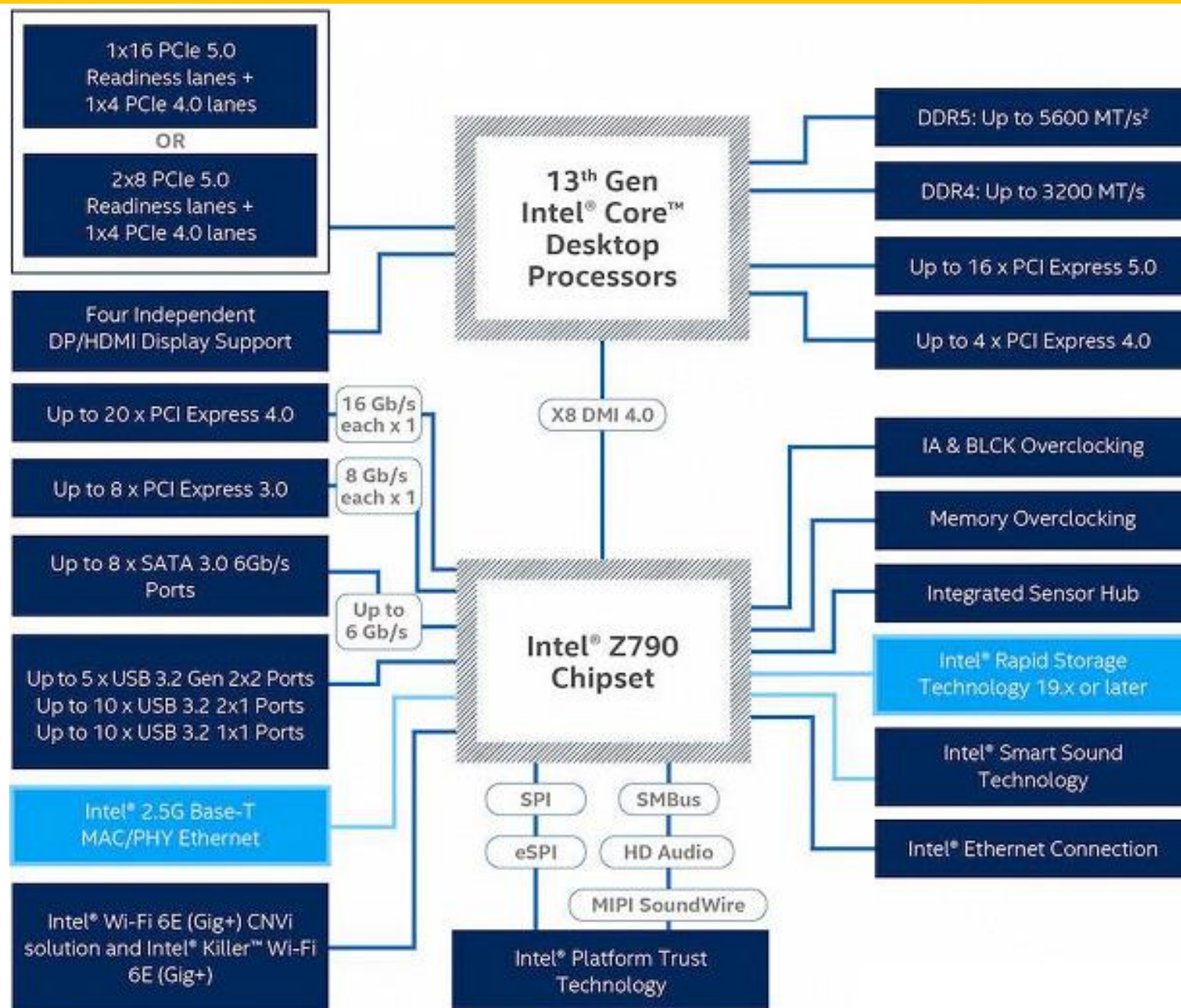


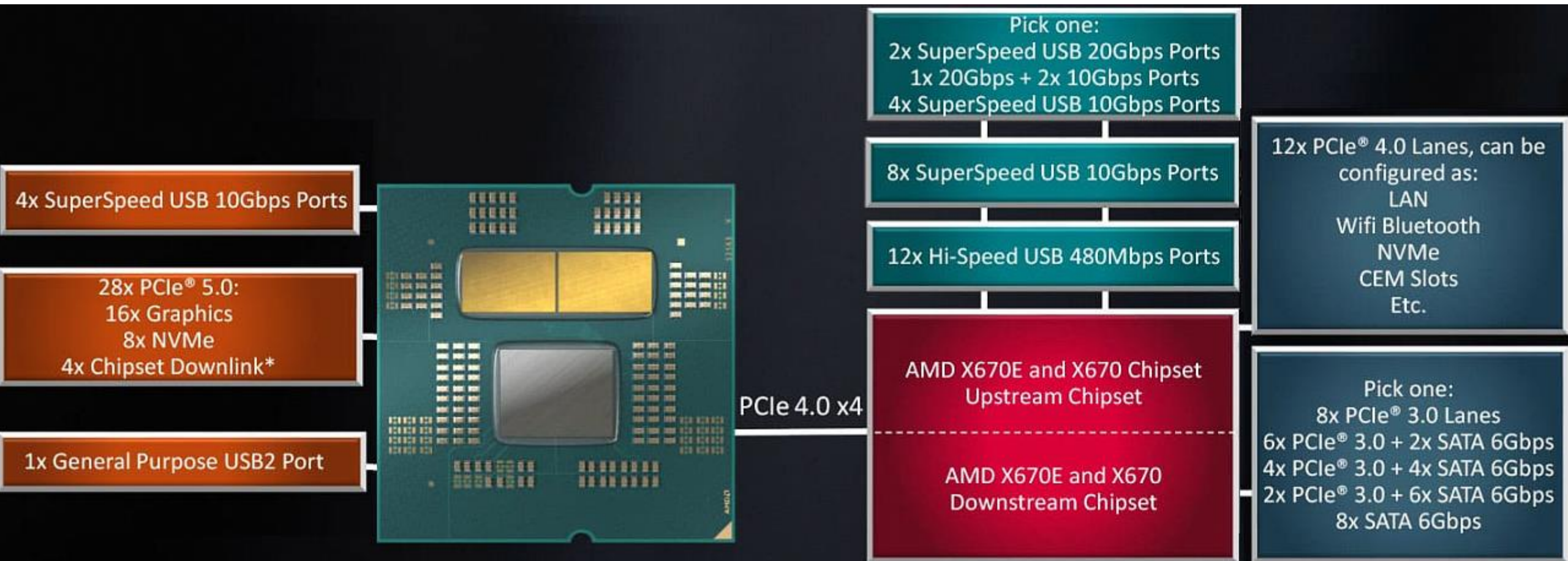


Традиционная структура ЭВМ

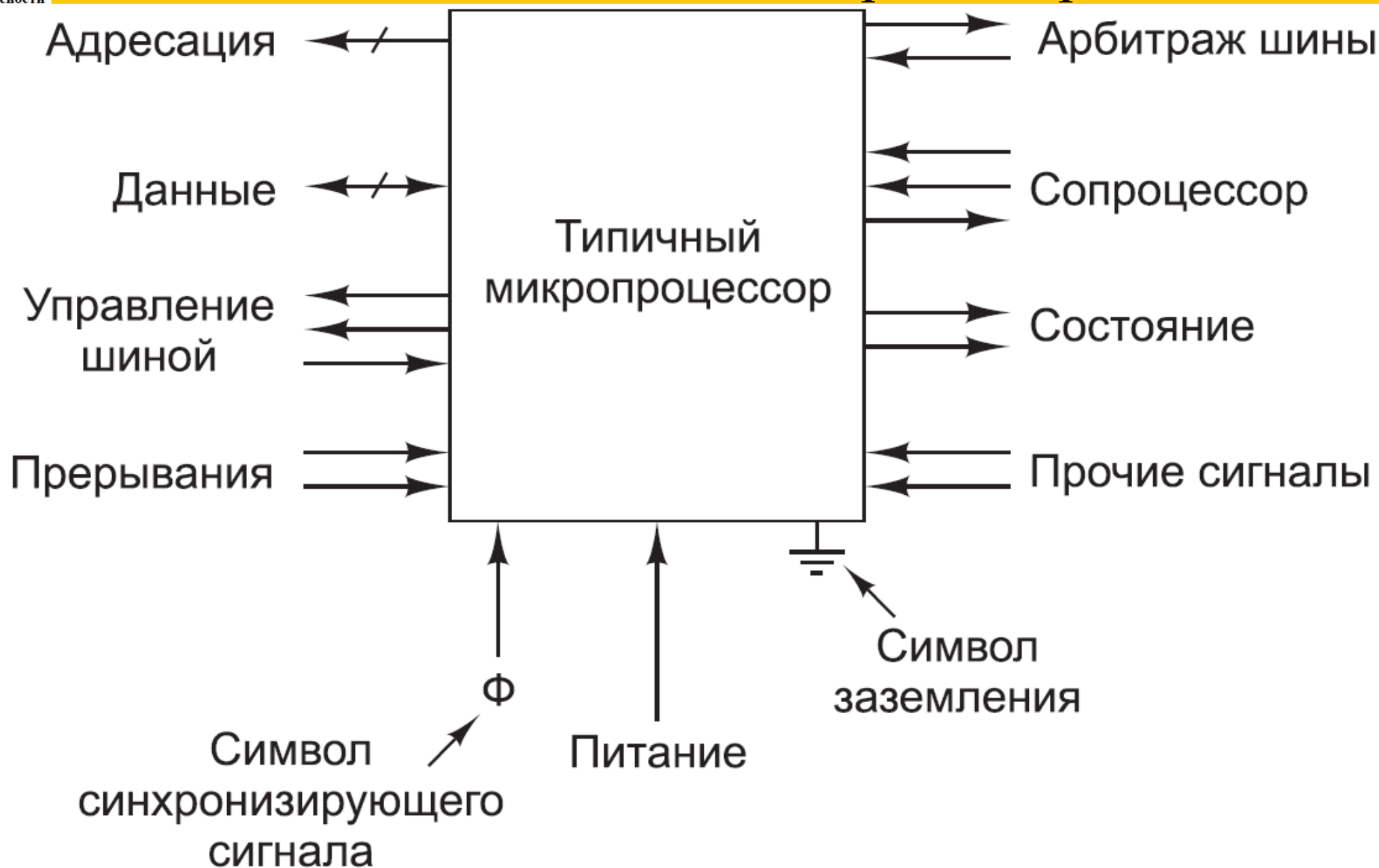


Чипсет Intel Z790

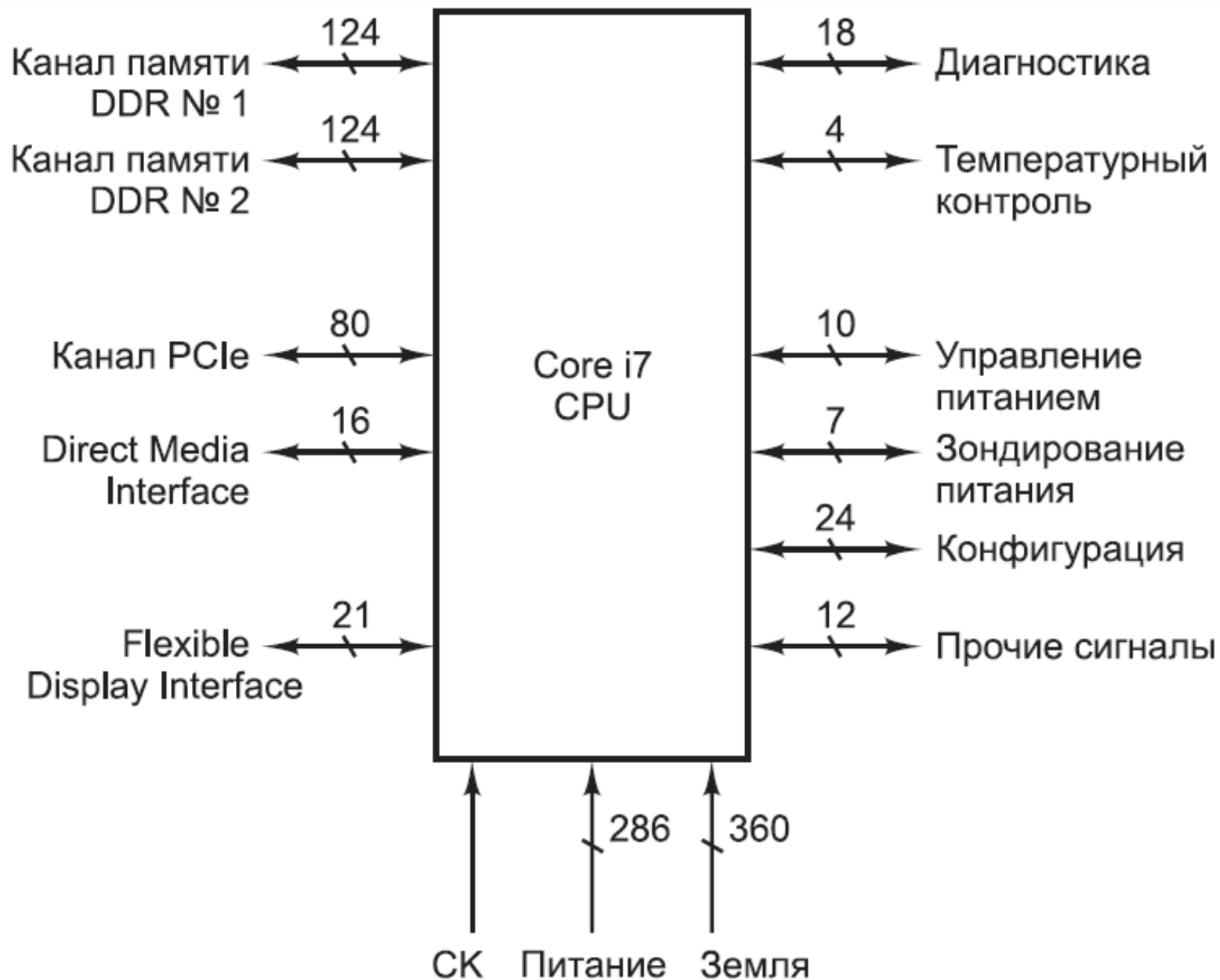




Выводы классического процессора



Выводы процессора Core i7





- **Команда** – элементарное действие, выполняемое ЦП
- **Порядок выполнения** (порядок исполнения, порядок вычислений) – это способ упорядочения инструкций программы в процессе её выполнения
- **Поток управления** – это последовательность, в которой выполняются команды программы.
- **Переход** – отклонение от естественного для применяемого способа записи порядка.
- **Исполнительный адрес** – номер ячейки памяти, по которому будет записан или считан операнд команды.
- **Командный цикл** – извлечение и выполнение одной команды



➤ ПРИНЦИП ДВОИЧНОГО КОДИРОВАНИЯ

Вся информация в ВМ кодируется с помощью двоичных сигналов.

Формат команд включает 2 части: код операции (КОП) и адресная часть (АЧ)

➤ ПРИНЦИП ПРОГРАММНОГО УПРАВЛЕНИЯ

Программа состоит из набора команд, автоматически выполняемых процессором друг за другом, в естественном порядке, существуют специальные команды, изменяющие порядок выполнения

➤ ПРИНЦИП ОДНОРОДНОСТИ ПАМЯТИ

Программы и данные хранятся в одной памяти и внешне неразличимы.

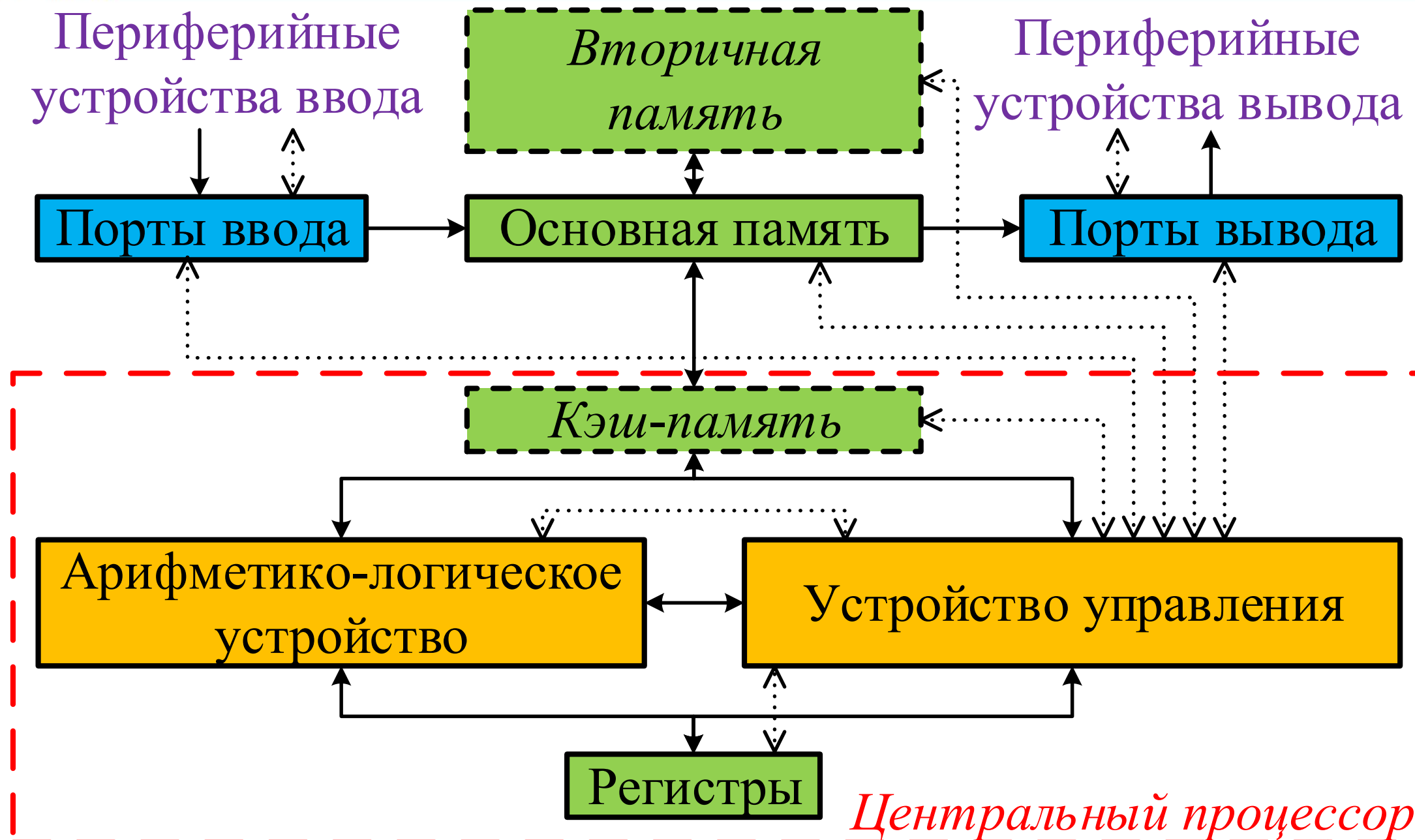
Над командами можно выполнять такие же действия, как и над данными

➤ ПРИНЦИП АДРЕСНОСТИ

Основная память состоит из пронумерованных ячеек. Процессору в произвольный момент времени доступна любая ячейка по её адресу



- Оперативная память (ОП) организована как совокупность машинных слов (МС) фиксированной длины или разрядности
- ОП образует единое адресное пространство, адреса МС возрастают от младших к старшим
- В ОП размещаются как данные, так и программы, причём в области данных одно слово, как правило, соответствует одному числу, а в области программы – одной команде
- Команды выполняются в естественной последовательности, пока не встретится команда управления
- ЦП может произвольно обращаться к любым адресам в ОП для выборки и/или записи в МС чисел или команд



Структура центрального устройства ВМ

Регистровая память

Регистр команды
Счётчик команд
Регистры адреса
Регистры числа
Регистр результата
Базисные регистры
Индексные регистры
Регистр состояния
Прочие регистры
Сумматор (аккумулятор)



АЛУ

Устройство управления

Число 2
Число 1
...
Команда k + 1
● Команда k
...

Программы

Оперативная память

КОП	ИР	БР	A1	A2	A3	...
Команда			Адресная часть			



- **АРИФМЕТИКО-ЛОГИЧЕСКОЕ УСТРОЙСТВО (АЛУ):**
 - выполнение арифметических и логических операций над данными.
- **УСТРОЙСТВО УПРАВЛЕНИЯ (УУ):**
 - синхронизации работы процессора с другими устройствами;
 - организация операций пересылки данных в процессор.
- **РЕГИСТРОВАЯ ПАМЯТЬ:**
 - хранение данных (в том числе служебных) для нескольких ближайших команд.
- **КЭШ-ПАМЯТЬ:**
 - временное хранение фрагментов ОЗУ (команд и данных) для выполнения нескольких команд.

Упрощённая схема процессора

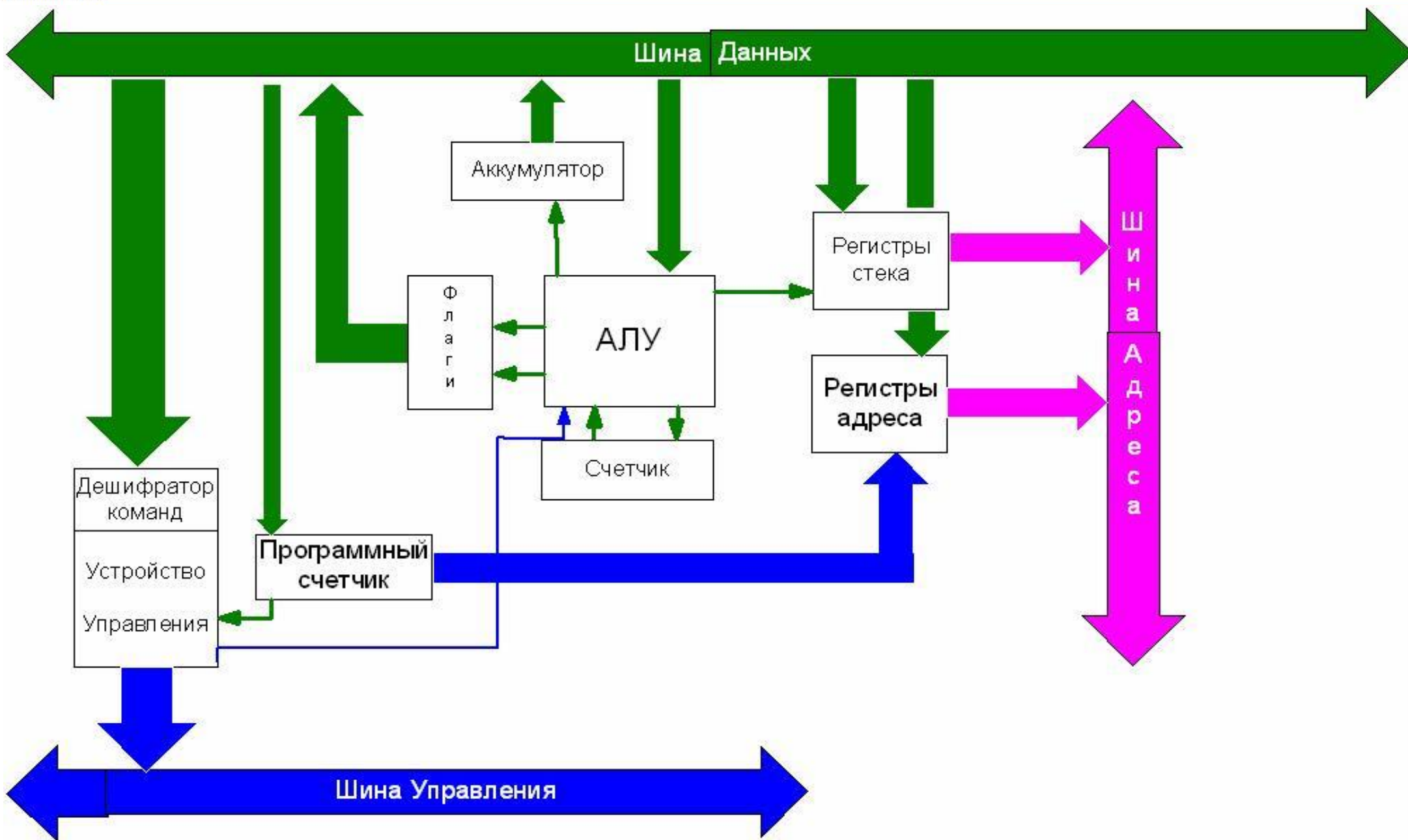
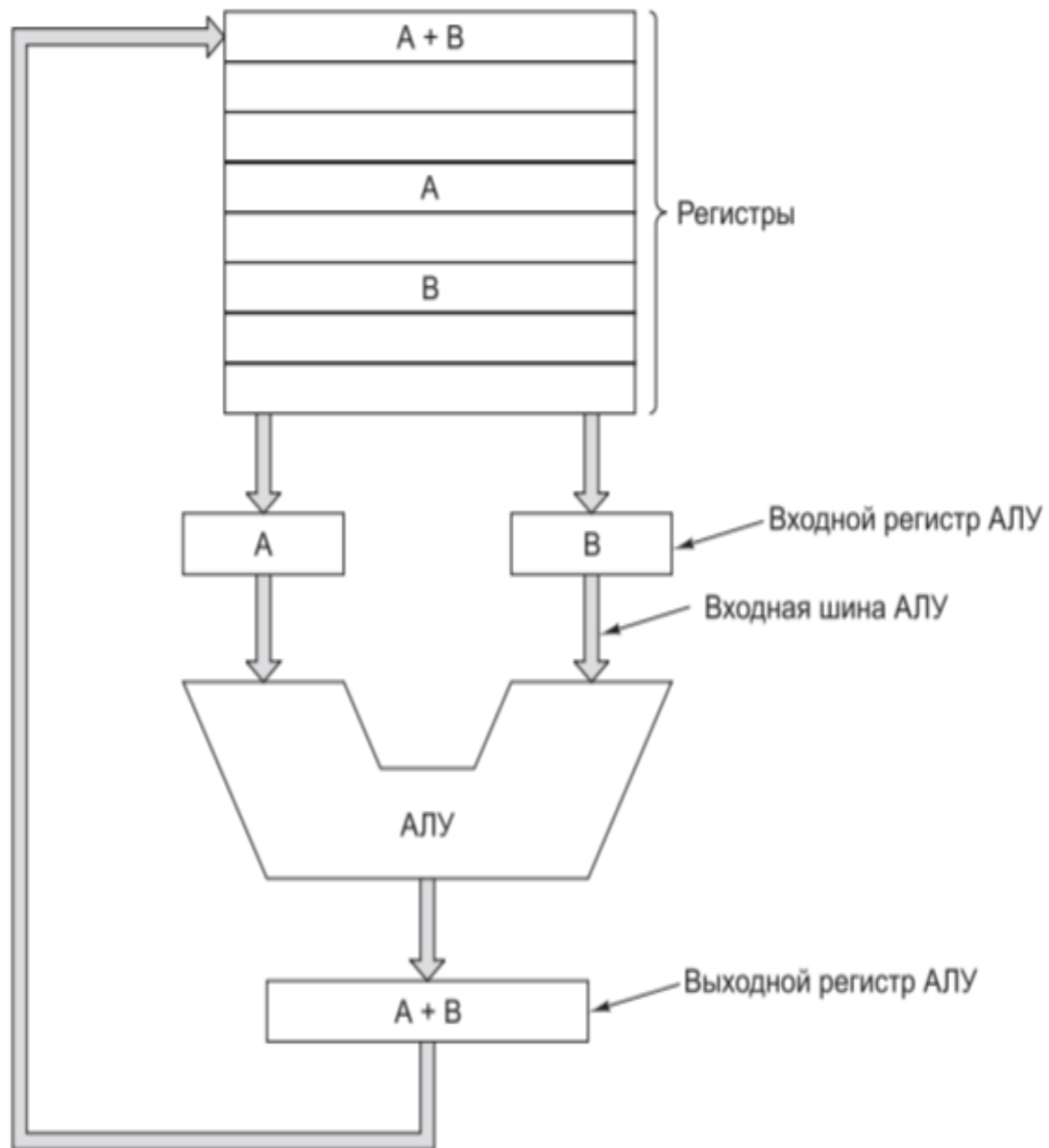
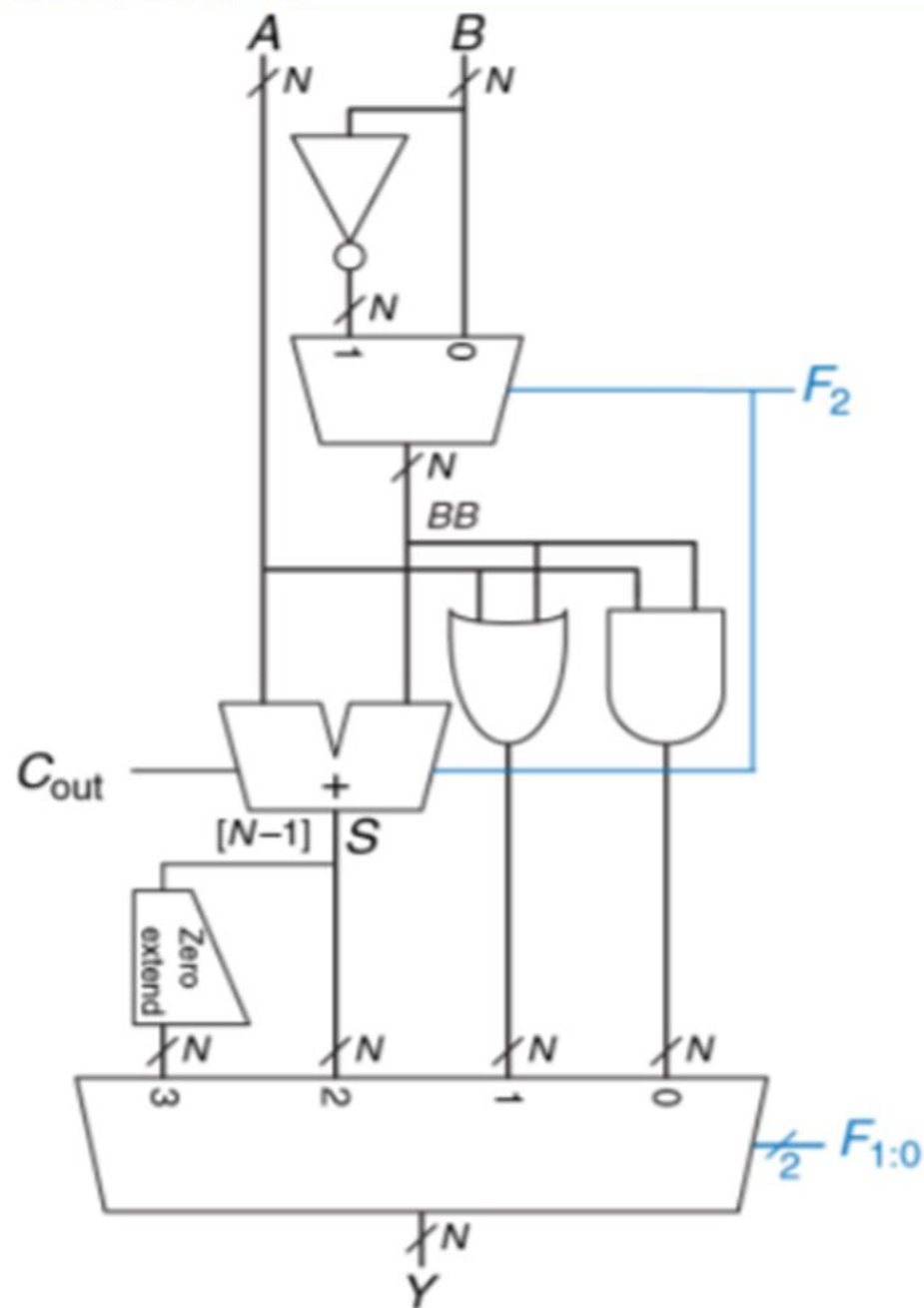


Схема арифметико-логического устройства



Пример узла АЛУ



$F_{2:0}$	Функция
000	A AND B
001	A OR B
010	A + B
011	—
100	A AND $\bar{B}$
101	A OR $\bar{B}$
110	A - B ($A + \bar{B} + 1$)
111	SLT A, B (Set if less than)

Пример одноразрядного узла АЛУ

INVA – инверсия A

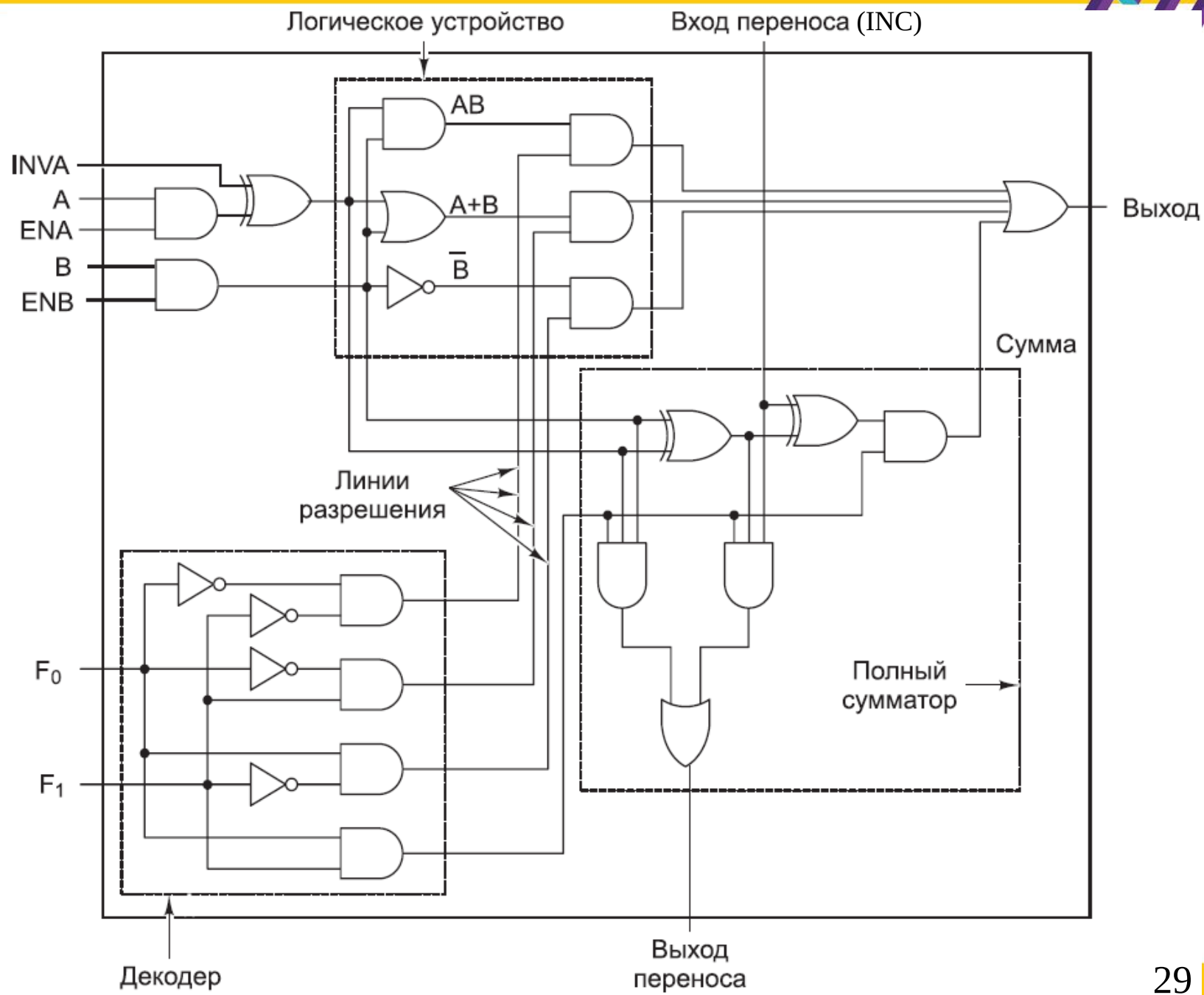
ENA – разрешение A

ENB – разрешение B

F₀ и F₁ – выбор действия

INC – перенос из предыдущего разряда

F0	F1	ENA	ENB	INVA	INC	Функция
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	not A
1	0	1	1	0	0	not B
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B – A
1	1	0	1	1	0	B – 1
1	1	1	0	1	1	–A
0	0	1	1	0	0	A И B
0	1	1	1	0	0	A ИЛИ B
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	0	1	0	–1





Устройства управления подразделяются:

➤ ***УУ с жёсткой (или схемной) логикой:***

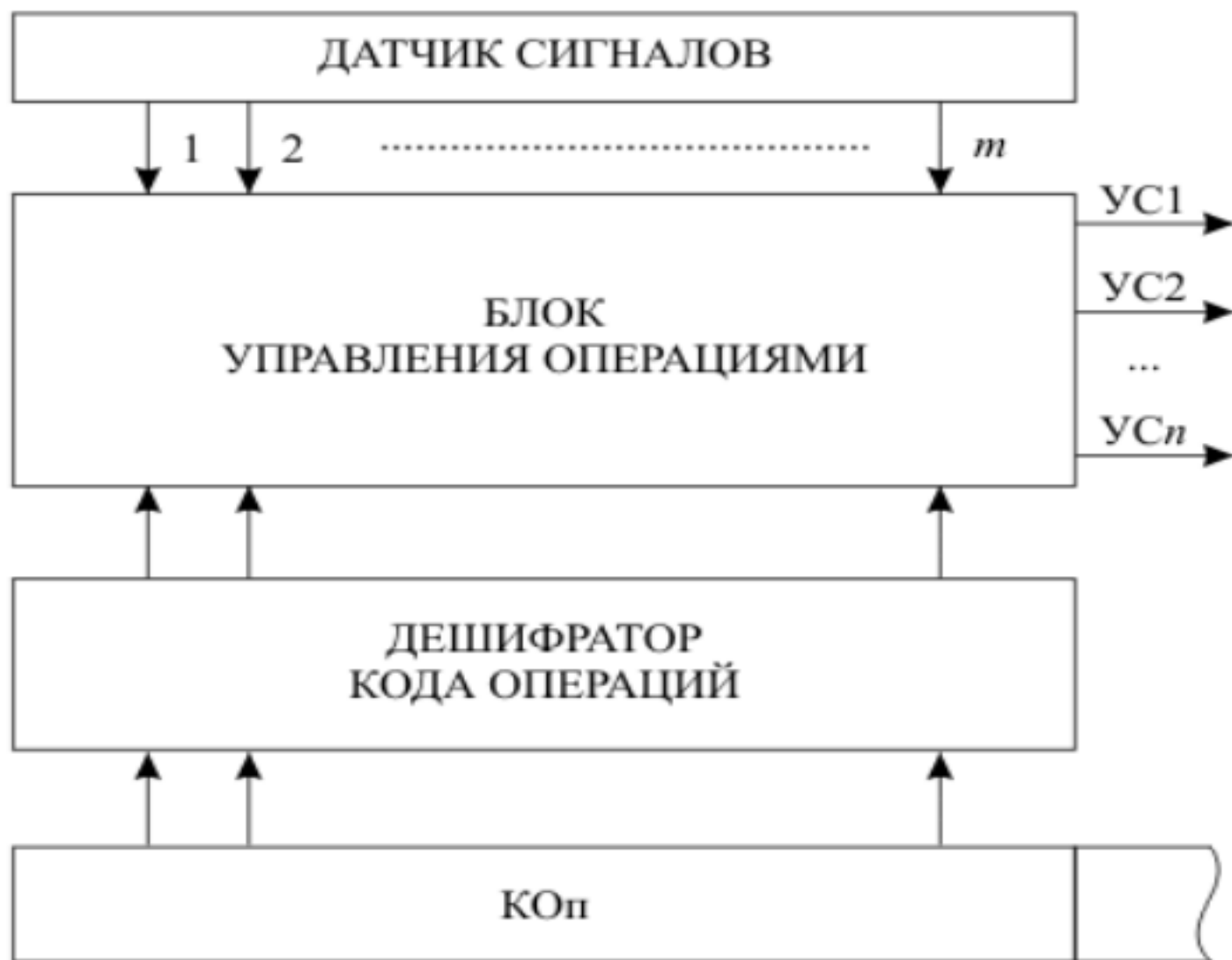
- ☐ сложная нерегулярная структура
- ☐ требуется специальная разработка для каждой системы команд
- ☐ необходима полная переработка при модификациях системы команд
- ☐ высокое быстродействие

➤ ***УУ с программируемой логикой (микропрограммные УУ):***

- ☐ легко настраивается на изменения в операционной части ЭВМ
- ☐ настройка заключается в замене микропрограммной памяти
- ☐ худшие временные показатели (по сравнению с УУ схемного типа)

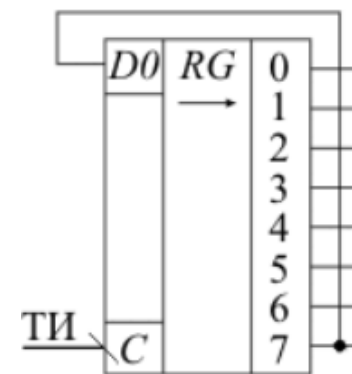
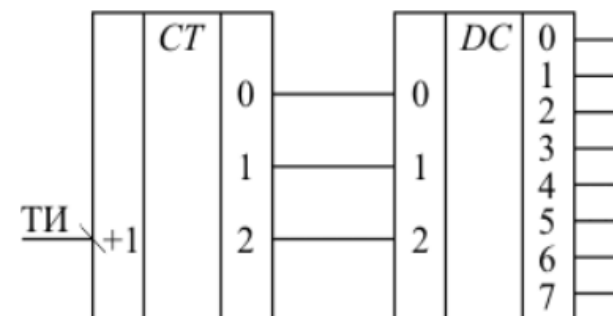


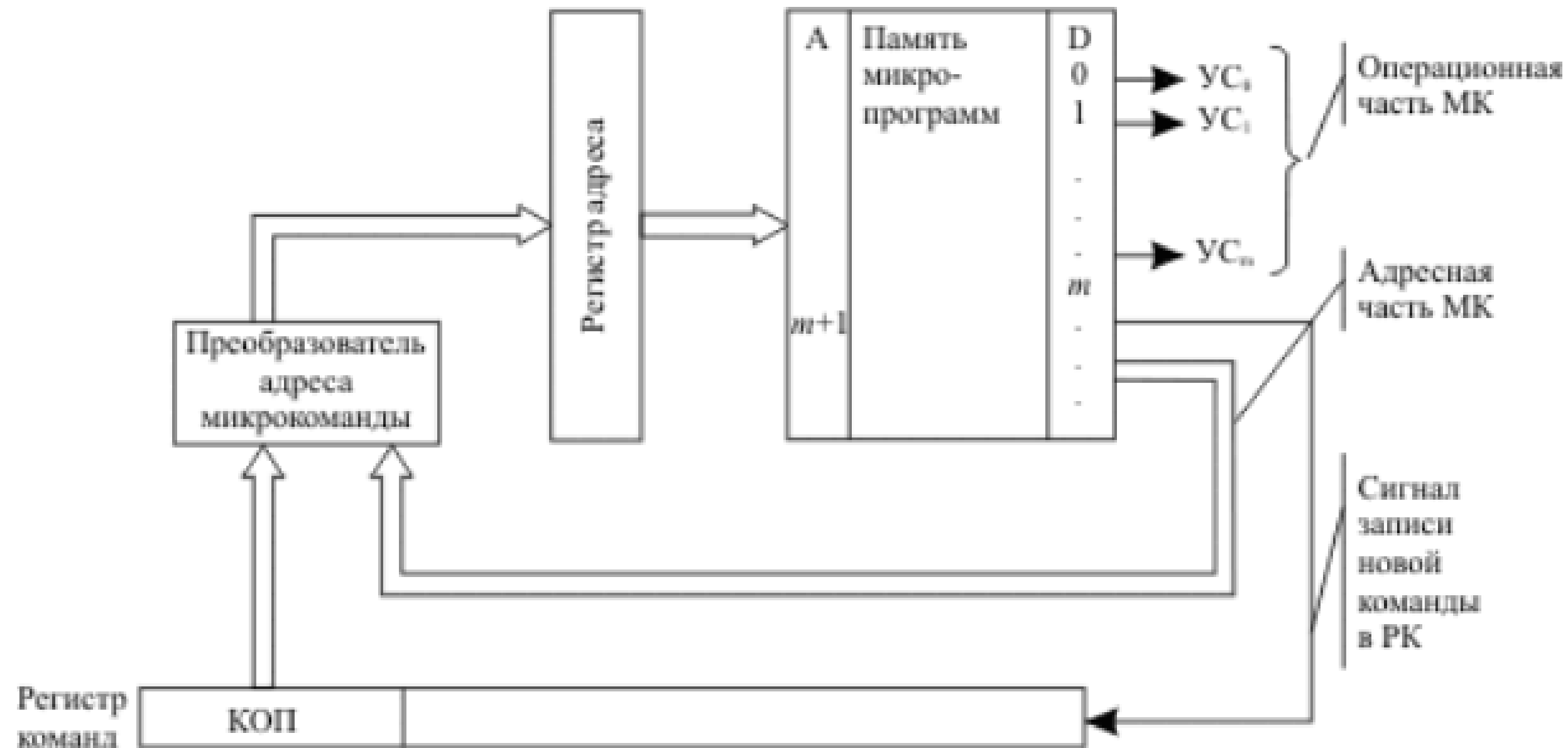
Регистр
команд



УС – управляющие сигналы

Варианты реализации датчика сигналов:

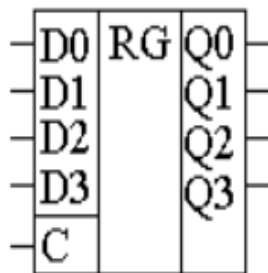
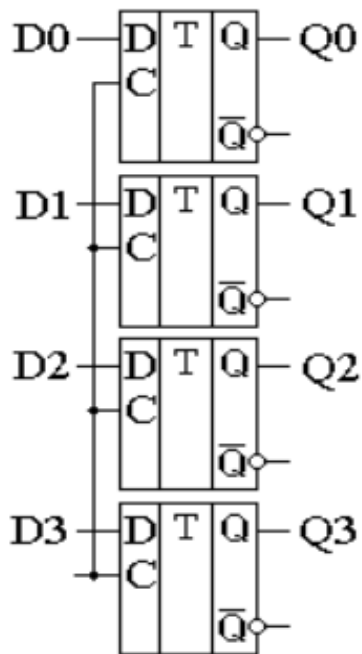




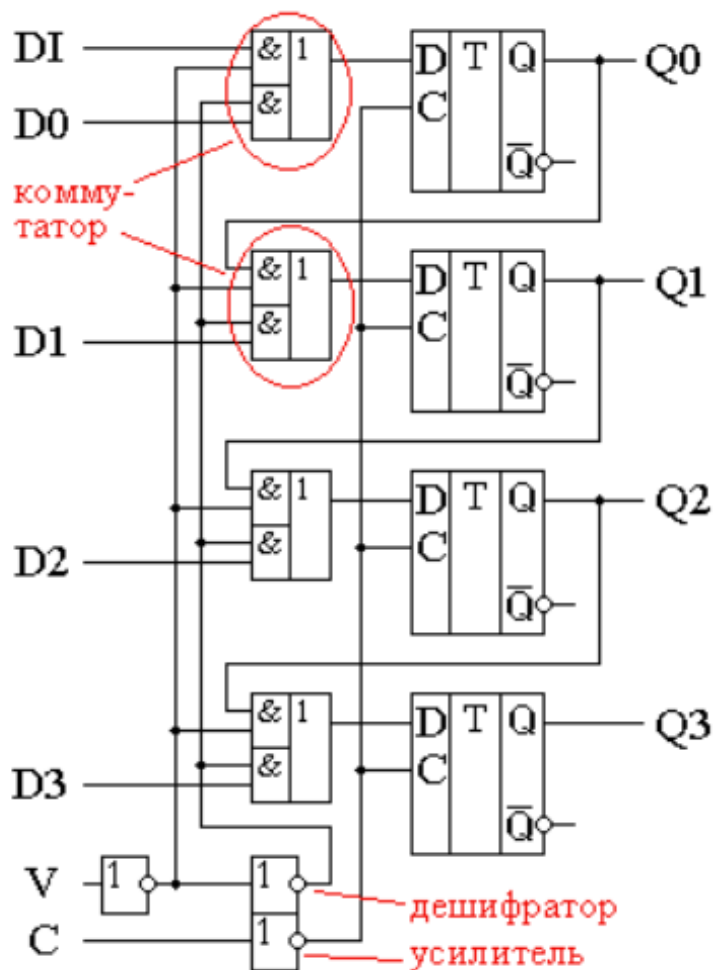


При построении регистров ЦП используется последовательное или параллельное соединение триггеров, обычно используются D-триггеры.

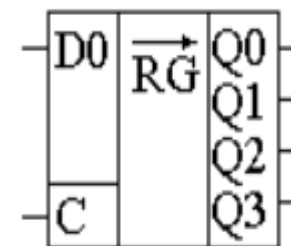
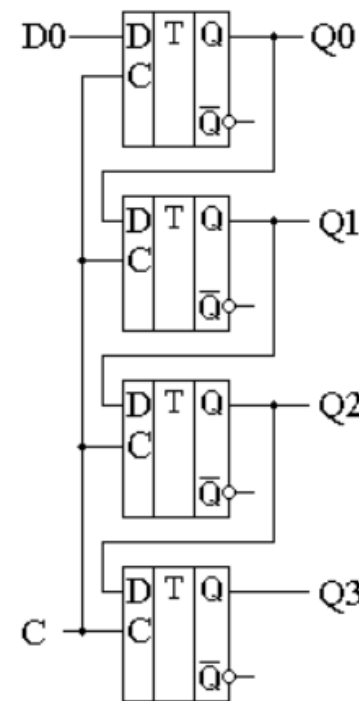
Параллельный



Универсальный



Последовательный



Регистры в архитектуре x86

	Имя	Разряды			Номер	Назначение регистра
		15-0	15-8	7-0		
Данные	EAX	AX	AH	AL	000	Выделенный сумматор, который используется во всех основных вычислениях
	ECX	CX	CH	CL	001	Универсальный счётчик циклов, который имеет специальную интерпретацию для циклов
	EDX	DX	DH	DL	010	Регистр данных, который дополняет сумматор и хранит данные, необходимые для операции, которая выполняется сумматором
	EBX	BX	BH	BL	011	База, используется как буферное ЗУ, но в режиме на 16 бит использовался как указатель (pointer)
Адреса	ESP	SP			100	Указатель стека, используется, чтобы хранить высший адрес стека
	EBP	BP			101	Указатель базы, используется, чтобы хранить адрес текущего фрейма стека. Может иногда применяться как буферное ЗУ
	ESI	SI			110	Индекс источника, обычно используемый в строковых операциях, чтобы загрузить данные из памяти на сумматор
	EDI	DI			111	Индекс назначения, обычно используемый в строковых операциях, чтобы записать данные из сумматора в память
	EIP	IP				Счётчик команд – указатель адреса команды
Сегменты	CS	CS				Сегмент команд
	DS	DS				Сегмент данных
	SS	SS				Сегмент стека
	ES	ES				Дополнительный сегмент

Флаги условий

CF	Carry flag	перенос	результат сложения не уложился в разрядную сетку, или при вычитании чисел без знака первое из них было меньше второго
OF	Overflow flag	переполнение	при сложении или вычитании целых чисел со знаком получился результат, по модулю превосходящий допустимую величину
ZF	Zero flag	ноль	результат команды оказался равным 0
SF	Sign flag	знак	в операции над числами со знаками получился отрицательный результат
PF	Parity flag	чётность	результат очередной команды содержит чётное количество двоичных единиц
AF	Auxiliary carry flag	доп. перенос	фиксирует особенности выполнения операций над двоично-десятичными числами

Флаги состояний

DF	Direction flag	направление	направление просмотра строк: при $DF = 0$ строки просматриваются «вперёд», при $DF = 1$ — «назад»
IF	Interrupt flag	прерывание	при $IF = 0$ процессор перестаёт реагировать на поступающие к нему прерывания, при $IF = 1$ блокировка прерываний снимается
TF	Trap flag	трассировка	после выполнения каждой команды процессор делает прерывание



Номер разряда	7	6	5	4	3	2	1	0
Имя флага	I	T	H	S	V	N	Z	C

$R \leftarrow \text{ADD } R_r, R_d$

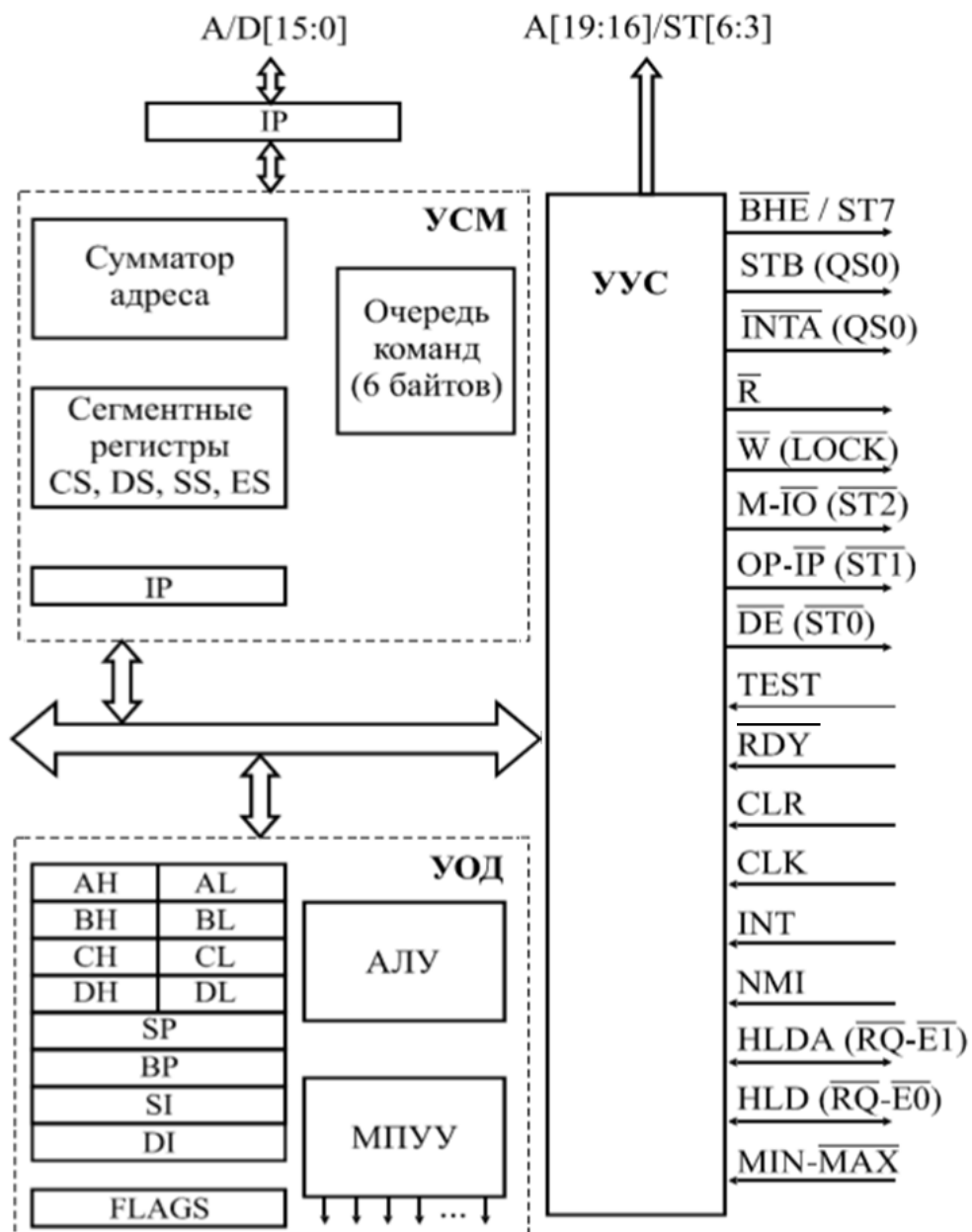
#	Имя флага	Описание
C	Carry flag	Флаг переноса: результат сложения не уложился в разрядную сетку или при вычитании чисел без знака первое из них было меньше второго: $C = Rd_7 \times Rr_7 + Rr_7 \times \overline{R_7} + \overline{R_7} \times Rd_7$
Z	Zero flag	Флаг нулевого результата. Устанавливается, если результат операции равен 0: $Z = \overline{R_7} \times \overline{R_6} \times \overline{R_5} \times \overline{R_4} \times \overline{R_3} \times \overline{R_2} \times \overline{R_1} \times \overline{R_0}$
N	Negative flag	Флаг отрицательного результата. Устанавливается, если старший разряд результата равен единице: $N = R_7$
V	Two's complement overflow	Флаг переполнения дополнения до двух. Устанавливается, если было переполнение разрядной сетки знакового результата: $V = Rd_7 \times Rr_7 \times \overline{R_7} + \overline{Rr_7} \times \overline{Rd_7} \times R_7$
S	Sign flag	Знак результата. Устанавливается, если результат отрицательный: $S = N \oplus V$
H	Half Carry flag	Флаг половинного переноса. Устанавливается, если во время выполнения операции был перенос из третьего разряда результата (используется в двоично-десятичной арифметике): $H = Rd_3 \times Rr_3 + Rr_3 \times \overline{R_3} + \overline{R_3} \times Rd_3$
T	Transfer bit	Разряд из регистра общего назначения может быть скопирован в Т с помощью BST, бит из Т может быть скопирован в разряд регистра с помощью команды BLD.
I	Global interrupt flag	Общее разрешение прерываний. Для разрешения каких-либо прерываний он должен быть установлен в единицу



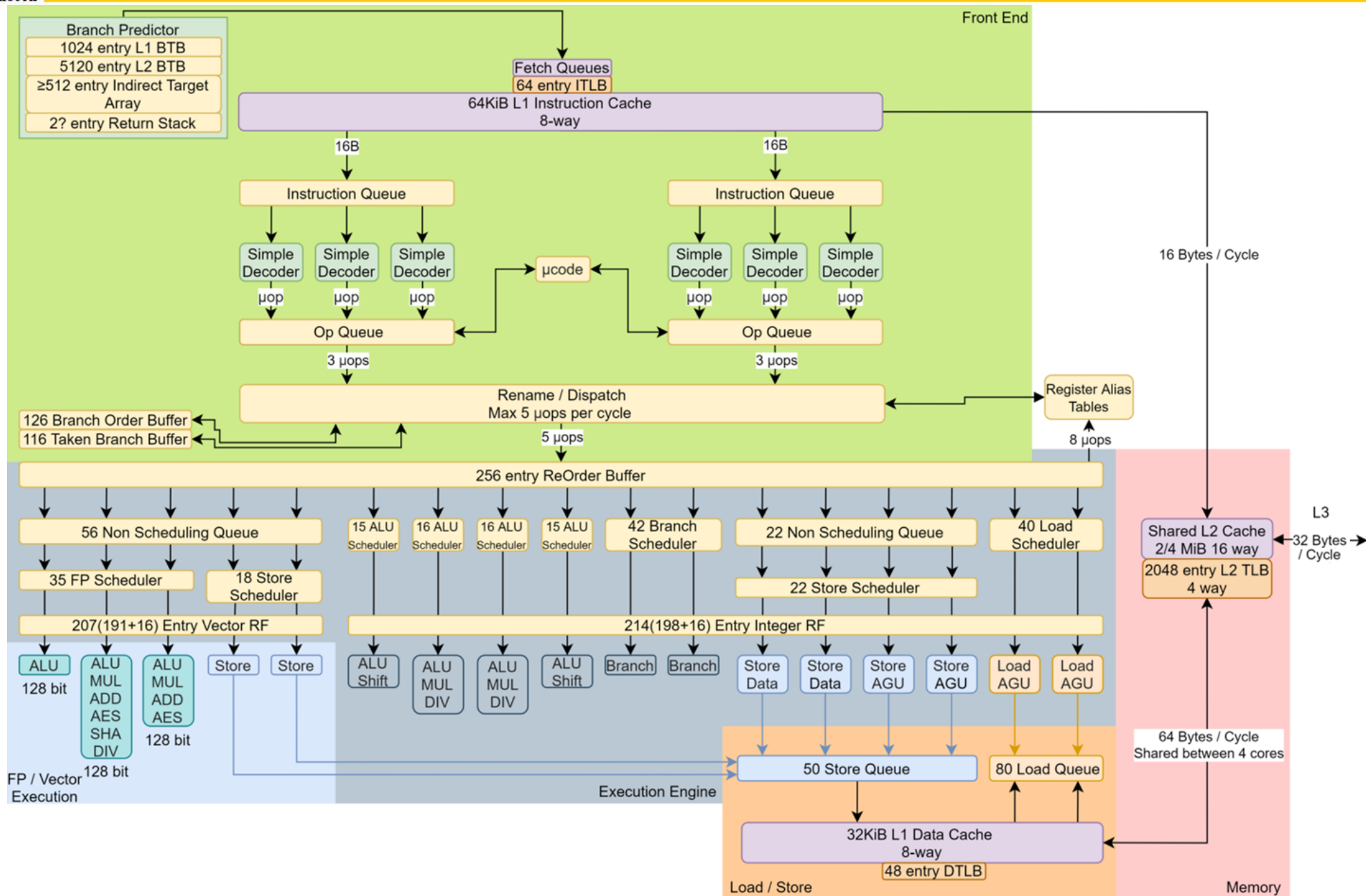
	Test	Boolean	Mnemonic	Complementary	Boolean	Mnemonic
Со знаком	$Rd > Rr$	$Z \cdot (N \oplus V) = 0$	BRLT*	$Rd \leq Rr$	$Z + (N \oplus V) = 1$	BRGE*
	$Rd \geq Rr$	$(N \oplus V) = 0$	BRGE	$Rd < Rr$	$(N \oplus V) = 1$	BRLT
	$Rd = Rr$	$Z = 1$	BREQ	$Rd \neq Rr$	$Z = 0$	BRNE
	$Rd \leq Rr$	$Z + (N \oplus V) = 1$	BRGE*	$Rd > Rr$	$Z \cdot (N \oplus V) = 0$	BRLT*
	$Rd < Rr$	$(N \oplus V) = 1$	BRLT	$Rd \geq Rr$	$(N \oplus V) = 0$	BRGE
Без знака	$Rd > Rr$	$C + Z = 0$	BRLO*	$Rd \leq Rr$	$C + Z = 1$	BRSH*
	$Rd \geq Rr$	$C = 0$	BRSH/BRCC	$Rd < Rr$	$C = 1$	BRLO/BRCS
	$Rd = Rr$	$Z = 1$	BREQ	$Rd \neq Rr$	$Z = 0$	BRNE
	$Rd \leq Rr$	$C + Z = 1$	BRSH*	$Rd > Rr$	$C + Z = 0$	BRLO*
	$Rd < Rr$	$C = 1$	BRLO/BRCS	$Rd \geq Rr$	$C = 0$	BRSH/BRCC
Общее	Carry	$C = 1$	BRCS	No carry	$C = 0$	BRCC
	Negative	$N = 1$	BRMI	Positive	$N = 0$	BRPL
	Overflow	$V = 1$	BRVS	No overflow	$V = 0$	BRVC

* Необходимо поменять местами аргументы

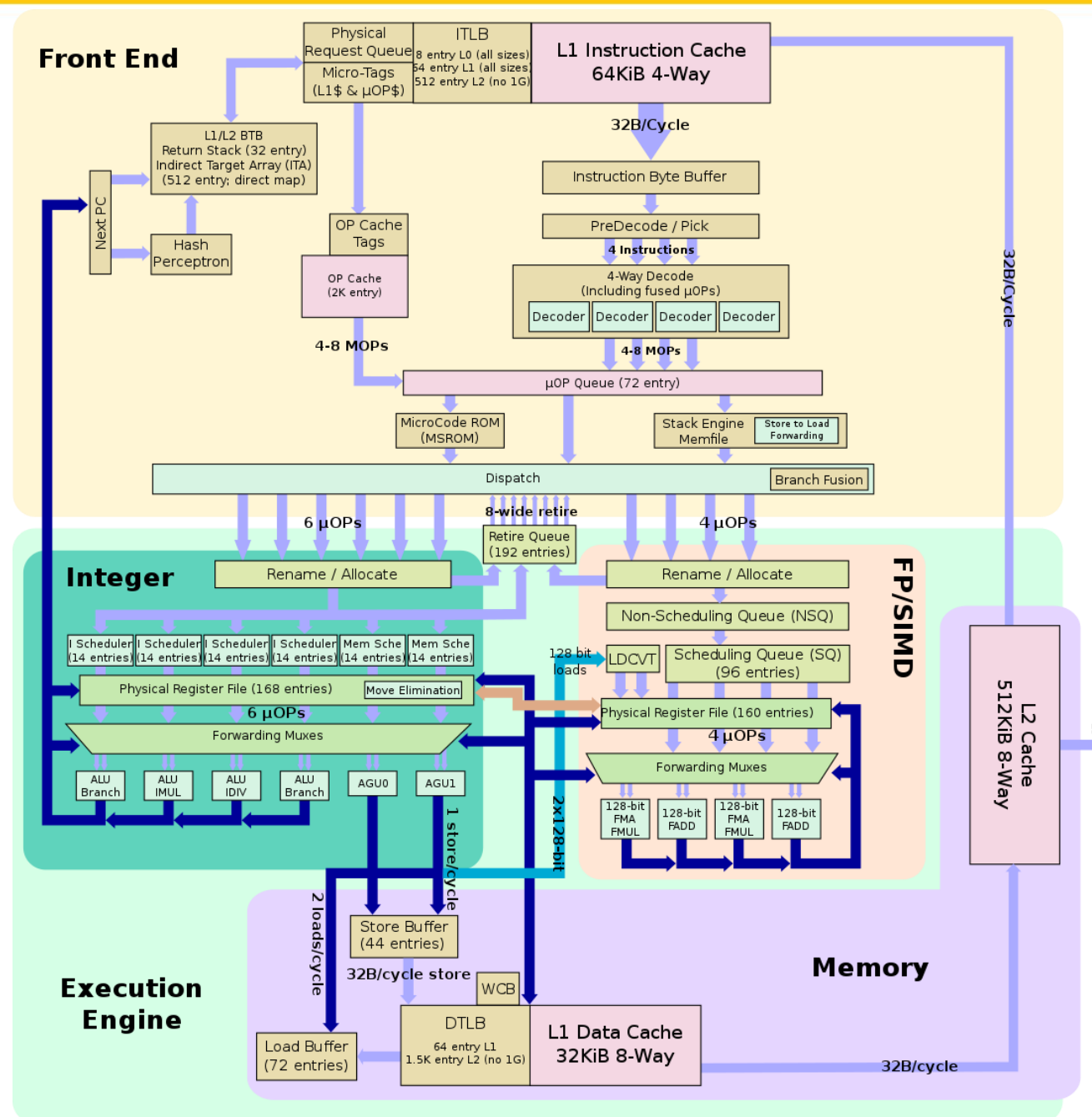
Структура микропроцессора (МП) i8086

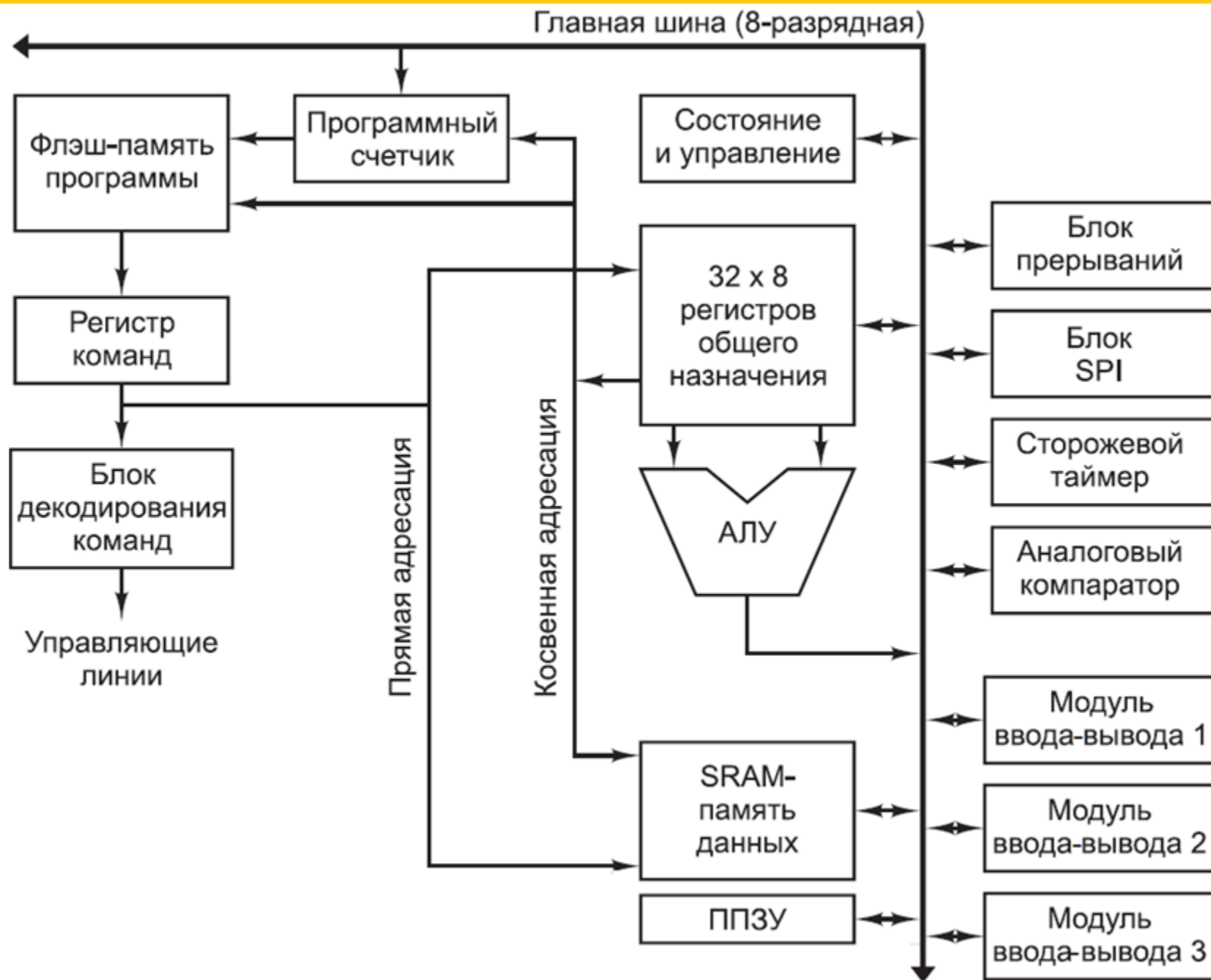


Сокращение	Пояснение
УСМ	Устройство связи с магистралью
УОД	Устройство обработки данных
МПУУ	Микропрограммное устройство управления
УУС	Устройство управления и синхронизации
Внешний вывод	
Описание	
A/D[15:0]	Младшие 0 – 15 разряды адреса/данные
A[19:16]/ST[6:3]	Старшие 16 – 19 разряды адреса/сигналы состояния
$\overline{BHE} / ST[7]$	Разрешение передачи старшего байта данных/сигнал состояния
$STB(QS0)$	Строб адреса (состояние очереди команд)
$\overline{R}$	Чтение
$\overline{W} (LOCK)$	Запись (блокировка канала)
$M-\overline{IO} (ST2)$	Память – внешнее устройство (состояние цикла)
$OP-\overline{IP} (\overline{ST1})$	Выдача/прием (состояние цикла)
$\overline{DE} (ST0)$	Разрешение передачи данных (состояние цикла)
$TEST$	Проверка
RDY	Готовность
CLR	Сброс
CLK	Тактовый сигнал
INT	Запрос внешнего прерывания
$\overline{INTA} (QS1)$	Подтверждение прерывания (состояние очереди команд)
NMI	Запрос немаскируемого прерывания
$HLD (\overline{RQ}-\overline{E0})$	Запрос ПДП (запрос/подтверждение доступа к магистрالي)
$HLDA (\overline{RQ}-\overline{E1})$	Подтверждение ПДП (запрос/подтверждение доступа к магистрالي)
$MIN-MAX$	Потенциал задания режима (min = 1, max = 0)



Примеры современных архитектур ЦП: AMD Zen







1. Извлечение из памяти содержимого ячейки, адрес которой хранится в счётчике команд, и размещение этого кода в регистре команды (чтение команды).
2. Увеличение содержимого счётчика команд на единицу.
3. Формирование адреса операндов.
4. Извлечение операндов из памяти/регистров.
5. Выполнение заданной в команде операции.
6. Размещение результата операции в памяти/регистрах.
7. Переход к п. 1.

Командный цикл процессора на архитектуре ATmega32

1. Передача флэш-памяти программ адреса текущей команды (из счётчика команд)
2. Увеличение счётчика команд на 1
3. Извлечение команды из флэш-памяти программ и запись в регистр команды
4. Декодирование команды
5. Извлечение адресов операндов (прямая адресация), возможно дополнение предопределённых старших разрядов
6. Извлечение операндов (непосредственная адресация), возможно выравнивание
7. Извлечение байта/бита из порта ввода-вывода
8. Передача операндов в АЛУ
9. Выполнение операции
10. Извлечение байта из ОЗУ (LD*, POP, RET*)
11. Обновление статусов (флаги в SREG и т.п.)
12. Запись результата в РОН
13. Запись результата в порт ввода-вывода
14. Запись результата в ОЗУ (ST*, PUSH, CALL*)
15. Запись результата в счётчик команд
16. Переход к пункту 1

